

# Frontend architecture

Ushbu PDF **JoshDeveloper**'ning "**Frontend arxitekturas**i" kursi uchun maxsus tayyorlangan. Uni faqat biz ulashgan insonlar va kursni sotib olganlar foydalanish huquqiga ega. Video va faylni ruhsatsiz tarqatish norozimiz.

Demak biz boshlaymiz, Bismillah! 😊

**Golden rule:** Architecture is not about tools, but mindset.

## ▼ Frontend architecture nima?



**Frontend architecture** - Frontend loyihalarni qurishda maxsus tamoyillar, strukturalar va best practice'lardan foydalangan holda **Scalable**, **Maintainable**, **Optimized**, **Consistent**, **Flexible**, **Testable** va **Business-aligned** dastur qurish.

1. **Scalable** - kengayuvchi. Ya'ni, dasturni kattalashtirish oson va jamoa kattalashsa ham ish sifati tushib ketmaydigan.
2. **Maintainable** - oson boshqariladigan. Ya'ni, dasturga yangi o'zgarish qo'shish yoki muommolarni to'g'irlash oson kechadi. Vaqt o'tishi dasturni boshqarishga katta salbiy ta'sir ko'rsatmaydi.
3. **Optimized** - optimallashtirilgan. Ya'ni, dasturni xotira tejashi, tez ochilishi va ishlashi, tez **build** bo'lishi, kod hajmi va shu kabi jihatlari o'ylangan va to'g'ri amalga oshirilgan.
4. **Consistent** - davomiy, izchil. Ya'ni, dastur ishlashida kutilmagan xatoliklar ro'y bermaydi, dastur davomiy va ishonchli quriladi. Bir maromda ishlaydi.
5. **Flexible** - moslashuvchan. Loyiha kodlari va komponentlar moslashuvchan ravishda yoziladi. Ko'pincha loyihada katta o'zgarishlar qilish qiyin kechmaydi. **Migration** ham oson bo'ladi.
6. **Business-aligned** - Loyiha biznes talablaridan kelib chiqqan holda quriladi va biznes xohishlarini amalga oshirish oson kechadi.
7. **Testable** - Test qilish oson.

## ▼ Connecting the "dots"



Connecting the dots in Frontend architectural parts

## ▼ ✓ Package manager va CLI

**Package manager** - Loyiha **dependency**'larini boshqarish va **script**'larni run qilish uchun ishlatiladi.

**Masalan:** `npm` `pnpm` `yarn`

**CLI** - esa **command-line interface** orqali loyiha yaratish, build qilish, test qilish imkonini beradi.

**Masalan:** `Angular CLI` , `Vite` , `Vue CLI` , `create-react-app`

## ▼ ✓ Builder, Compiler va Bundler

**Builder:** Kodingizni **build** qiladi — ya'ni, **compile/transpile** (*Angular IVY, Babel, Vue compiler*), **minify**, **tree-shake** qiladi.

**Bundler:** Fayllarni birlashtirib **final bundle** yaratadi.

**Masalan:** Webpack, esbuild, Rollup, Vite.

## ▼ ✓ Architecture strategy + Workspace management

**Architecture strategy** - loyiha arxitekturasi.

**Masalan:** Monolith, Micro Frontend arxitektura (va bularning ichidagi barcha kichik arxitekturaviy qarorlar)

**Workspace management** - bir Repository'da ichidagi barcha loyihalarni qulay va oson boshqarish. Asosan qo'shimcha **Tool**'lar ishlatiladi.

**Masalan:** Nx (Angular, React), Turborepo, pnpm workspaces

## ▼ ✓ Core Framework

UI yaratish uchun asosiy framework yoki kutubxona.

- **Angular: Full-featured framework** — routing, dependency injection (DI), forms, pipes, HTTP va boshqalar hammasi ichida.
- **React:** Faqat **View** layer uchun **minimal** kutubxona. **JSX** asosida ishlaydi. Routing, state, va boshqalar uchun qo'shimcha kutubxonalarni siz tanlaysiz.
- **Vue: Progressive framework** — template-based sintaksisga ega, o'rganish oson.

## ▼ ✓ State Management

**State Management** — Ma'lumotlarni saqlash, uzatish, sinxronlash va o'zgartirish uchun qo'llaniladi. Ko'pincha qo'shimcha **Tool**'lardan foydalaniladi.

- **Angular:**
  - `Signals` – reactive primitives (yangi usul)
  - `RxJS` – reactive stream management
  - `NgRx` – redux-style centralized store
  - `NgRx/signals` - Signal-based store
- **React:**

- `Redux` – centralized state container
- `Zustand` – minimal global state manager
- `useState`, `useReducer` – built-in hooks for local state

- **Vue:**

- `Pinia` – official state management library
- `reactive`, `ref`, `computed` – core reactivity API

## ▼ **API Layer (Data Fetching)**

Backend bilan muloqot qilish, HTTP so'rovlar va ular bilan ishlash

- **Angular:**

- `HttpClient` – Angular'ning rasmiy HTTP service'i
- `RxJS operators` – so'rovlarni boshqarish (retry, switch, cancel va hokazo)

- **React:**

- `React Query / TanStack Query` – data fetching, caching, background updates
- `axios` or `fetch` – HTTP requests

- **Vue:**

- `Axios` – eng ko'p ishlatiladigan HTTP client
- `Vue Query` – Vue uchun `TanStack Query`'ga o'xshash library

## ▼ **Folder structure**

`Folder` (papka) strukturasini qismlarga va modullarga bo'lish.

Masalan: `core/`, `shared/`, `features/`, `lib/`, `pages/`, `services/`

## ▼ **Routing**

Ilova sahifalari va URL navigatsiyasini boshqarish.

**Angular:** `@angular/router`

**React:** `react-router`, Next.js routing

**Vue:** `vue-router`, Nuxt routing

## ▼ **Security layer**

Frontend darajasidagi xavfsizlik:

- XSS oldini olish (`sanitization`, `DOMPurify`)
- Route guards
- Tokenlar bilan ishlash (`JWT`, `CSRF` va hkz)
- Role-based UI ko'rsatish

## ▼ **Design System**

Ilova UI komponentlari, ranglar, spacing, fontlar — hammasi yagona tizimda boshqariladi.

**Angular:** Angular Material, Tailwind

**React:** Chakra, Material UI, Tailwind

**Vue:** Vuetify, UnoCSS, Tailwind

## ▼ **Quality gates va Testing**

**Quality gates** — kod sifati nazorati uchun ishlatiladigan texnologiyalar:

- ESLint, Prettier
- Husky hooks (masalan pre-commit checks)
- Test coverage (Jest, Vitest, Cypress)
- Static analysis (SonarQube)

**Testing** — loyiha kodi va ishlashini tekshirish.

**Testing turlari** — Unit, Integration, E2E

- **Angular:** Jasmine+Karma/Jest + TestBed
- **React/Vue:** Jest + Testing Library

## ▼ **CI/CD**

Kodni build qilish, test qilish va avtomatik ravishda deploy qilish jarayoni.

Har bir push yoki merge'dan so'ng avtomatik ishlaydi.

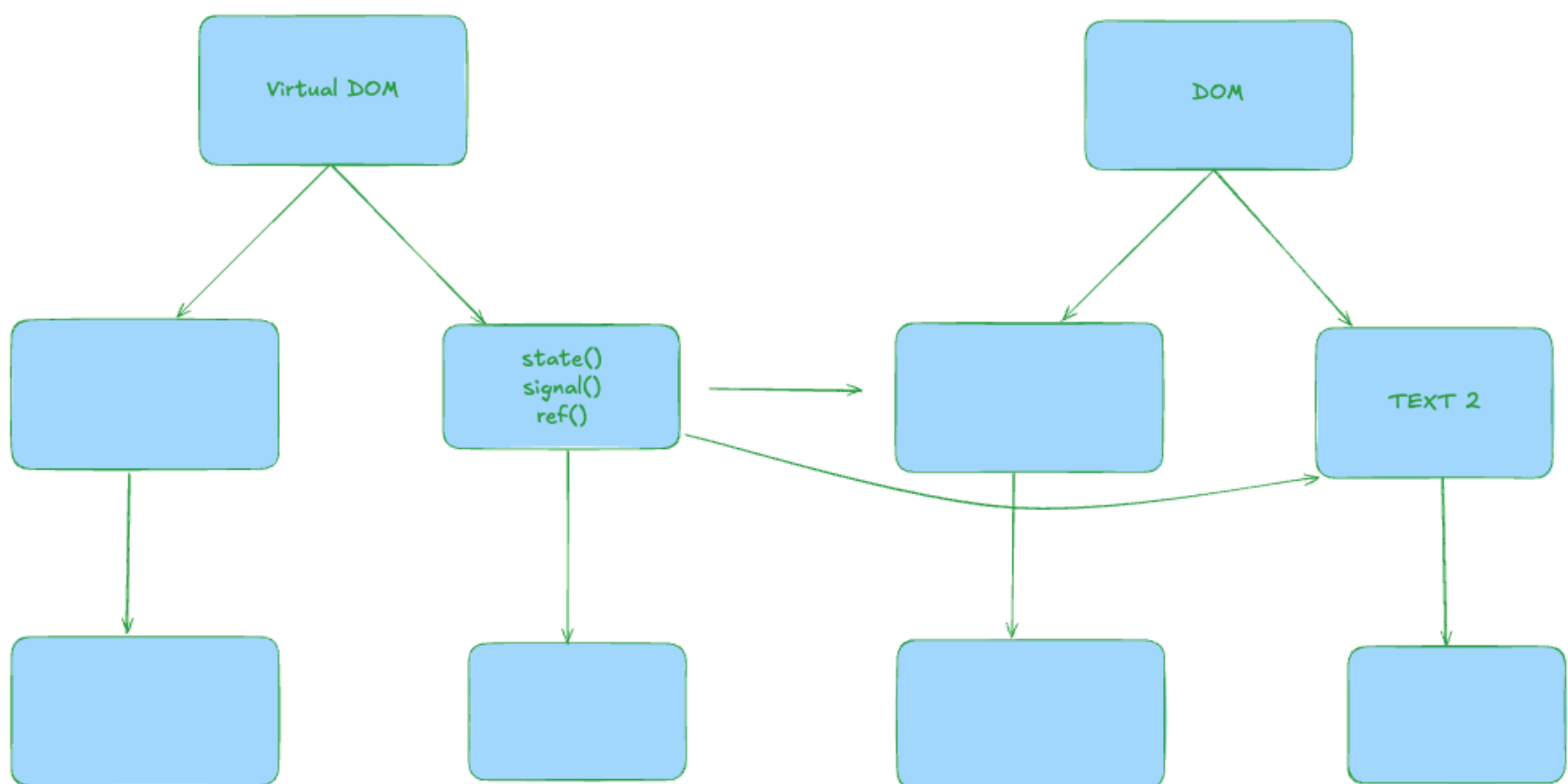
- **Asosan CI/CD'da yoziladigan qadamlar:**
  - Code format checking
  - Linting
  - Testing
  - Build va Deploy
- **Tools:** GitHub Actions, GitLab CI, Bitbucket pipelines, Vercel, CircleCI, Netlify, Azure DevOps, Jenkins
- **Repo:** <https://github.com/josh-developer/CI-CD-practice>

**Task:** O'z loyihangiz "**Dot**"larini aniq texnologiya va yechimlar bilan ta'riflang.

## ▼ **Reactivity**

Blabla

# Reactivity - O'zgaruvchanlik



## ▼ **Frontend** arxitekturasida ko'p uchraydigan xatolar

### ▼ Noto'g'ri **architecture** tanlash



**Definition:** Hamma loyiha harxil ekanini hisobga olib, barchasiga individual yondashish zarur. Ba'zi hollarda loyihalarni ajratish kerak emas bo'lgan joyda **Micro-frontend** qilishadi, yoki loyihalarga ajratish kerak bo'lgan joyda hamma narsani bir joyda yozishadi va loyiha kattalashib ketadi.

**Result:** O'ziga mos bo'lmagan arxitekturaga ega loyihalar bir kun kelib qayta yozilishga majbur bo'ladi. Chunki, bunday loyihalar ko'pincha **Maintainable**, **Scalable** va **Flexible** bo'lmaydi.

## ▼ Layer'larga ajratmaslik (UI, Business, Data)



**Definition:** Loyihani 3 ta asosiy **layer**'ga ajratmaslik bu eng keng ko'rilgan xatolardan. Bunda, ko'pincha **UI component**'lar biznes logikani ham o'zida jamlaydi, yoki teskarisi. **State management** ham noto'g'ri foydalaniladi.

**Result:** Loyiha **Scalable**, **Consistent**, **Optimized**, **Flexible** va **Maintainable** bo'lmay qoladi.

## ▼ SoC'ga amal qilmaslik



**Definition:** SoC - **Seperation of Concerns**. Har bir qism o'z vazifasini qilishi kerak. Unday bo'lmagan loyihalar **Spaghetti**'ga aylanib ketadi.

**Result:** Loyiha **Scalable**, **Flexible** va **Maintainable** bo'lmay qoladi.

## ▼ Domain-oriented fikrlash yo'qligi, folder-structure xato qurilishi



**Definition:** Loyihani faqat texnik tushuncha va fikrlash bilan qurish. Biznesga moslab struktura qilmaslik va **feature**'larga bo'lmaslik (**Feature** - asosan **UI**, **Logic**, **State** va **API**'dan iborat bo'lishi kerak).

**Result:** Loyiha **Business-aligned**, **Flexible** va **Maintainable** bo'lmay qoladi.

## ▼ Over-engineering qilish



**Definition:** Kerak bo'lmagan joylarda juda qiyin logikalar qilish. Osonlashtiraman deb, qiyinlashtirish. Masalan, 2 ta harxil sahifani o'xshash joylari ko'p deb, bitta sahifani **dynamic** qilishga urunish.

**Result:** Loyiha **Maintainable** va **Scalable** bo'lmay qoladi.

## ▼ Shared-state'larni me'yoridan ortiq ishlatish



**Definition:** Global turgan **state**larni me'yoridan ortiq ishlatish. Ko'p hollarda **local state** yetarli bo'ladi.

**Result:** Loyihada **state** boshqaruvi og'irlashadi, ba'zan **memory-leak**lar yuzaga chiqadi. Bunday loyiha ko'pincha **Maintainable**, **Testable** va **Consistent** bo'lmaydi.

## ▼ Tight coupling qilish



**Definition:** Loyihadagi komponentlar va logikalar bir biriga qattiq bog'lanishi — **tight coupling** deyiladi. Masalan, **feature-1** o'z ichida **feature-2**'ni komponentlarini ishlatishi. Yoki, **drag-and-drop component** o'z ichida **feature-3 API** so'rovlarini ishlatishi.

**Result:** Loyiha **Maintainable**, **Scalable**, **Testable**, **Flexible** va **Consistent** bo'lmay qoladi.

## ▼ Initial load haqida o'ylamaslik



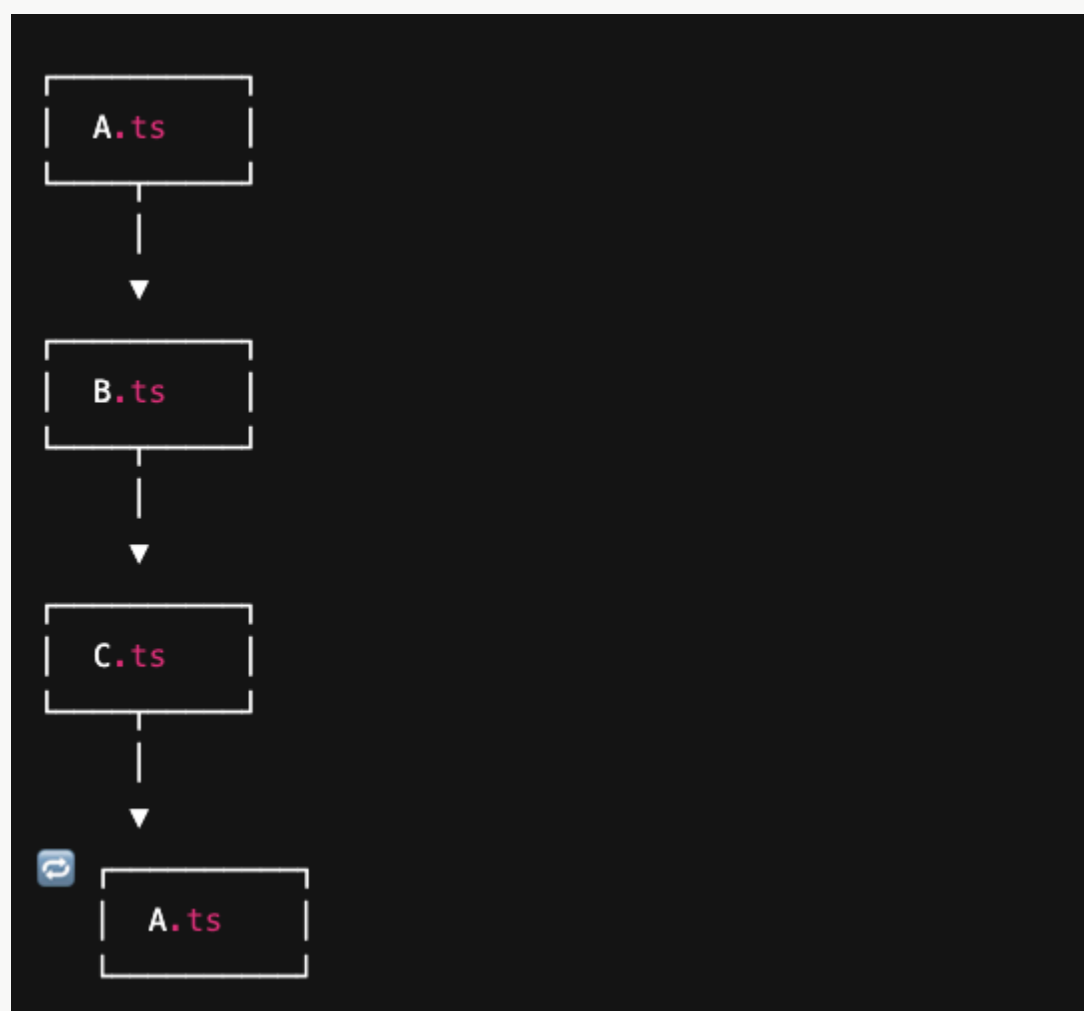
**Definition:** **Initial load** — bu loyiha ilk marta yuklanishdagi jarayon. Masalan loyiha modullari va ikkinchi darajali rasmlarni **lazy load** qilmaslik asosiy **bundle-size**ni hajmini oshirishga sabab bo'ladi. Shu kabi harxil ta'sirlar mavjud.

**Result:** Loyiha **Optimized** bo'lmay qoladi.

## ▼ Circular dependency



**Definition:** ikki yoki undan ortiq modullar bir biriga bog'lanib qolganda sodir bo'ladigan muammo **Circular dependency** deyiladi. Masalan,



**Result:** Loyiha sinadi. Chunki **dependency**lar o'zaro **Loop** yaratishgan.

## ▼ Fat components



**Definition:** bir **component** ichida ham **API**, ham **UI**, ham **State** va hokazolar yozilsa bu **component** — **Fat component** deyiladi.

**Result:** Loyiha **Flexible**, **Maintainable** va **Scalable** bo'lmay qoladi.

## ▼ Memory leaks





**Definition:** **Memory leaks** - bu loyiha qaysidir maqsadi uchun **RAM**'dan joy oladi va uni bo'shatmaydi.

Masalan,

1. Event Listeners
2. Global variables
3. Timers (setTimeout, setInterval)
4. Subscriptions
5. etc...

**Result:** Loyiha **Optimized**, **Consistent** bo'lmay qoladi.

## ▼ **Architecture turlari**

### ▼ **Monolithic Frontend (Modular va Spaghetti monolith)**



**Monolithic frontend** — bu bitta, yaxlit kod bazasida yozilgan frontend arxitekturası bo'lib, barcha funksiyalar, sahifalar va komponentlar bir joyda joylashgan bo'ladi. **UI**, **Logic** va **State** boshqarish barchasi shu yagona tizimda bajariladi. **Build/Deploy** esa bitta bo'ladi.

**Ikkita asosiy turi mavjud:**

- **Modular Monolith** – Loyiha modullarga ajratilib yoziladi. Barcha narsa aniq vazifasiga ega va boshqa logikalardan ajratilgan bo'ladi. To'g'ri yozilsa **Maintainable**, **Flexible**, **Business-aligned** va **Scalable** bo'ladi.

Katta loyihalar uchun ayni muddao. Misol:

```
src/
├── features/
│   ├── auth/
│   │   ├── components/
│   │   ├── services/
│   │   ├── hooks/
│   │   └── index
│   ├── dashboard/
│   │   ├── components/
│   │   ├── services/
│   │   ├── hooks/
│   │   └── index
│   └── shared/
│       ├── components/
│       ├── utils/
│       ├── constants/
│       └── index
├── routes/
│   └── app-routes
├── app/
│   ├── app
│   └── main
└── assets/
```

Modular monolith

- **Spaghetti Monolith** – Tuzilishi noaniq, komponentlar va funksiyalar bir-biriga qattiq bog'langan(**Tight coupling**) bo'ladi. **Readability** qiyin bo'ladi. **Maintainable**, **Flexible**, **Business-aligned** va **Scalable** bo'lmaydi.

Juda kichik loyihalar uchun ishlatsa bo'ladi. Misol:

```
src/
├── components/
│   ├── Button/
│   │   ├── Button.js
│   │   └── Button.css
│   ├── Navbar/
│   │   ├── Navbar.js
│   │   └── Navbar.css
│   ├── LoginForm.js
│   ├── DashboardWidget.js
│   ├── UserProfile.js
│   └── ChartComponent.js
├── services/
│   ├── api.js
│   ├── authService.js
│   └── userService.js
├── utils/
│   ├── helpers.js
│   └── validators.js
├── styles/
│   ├── main.css
│   └── theme.css
├── pages/
│   ├── HomePage.js
│   ├── DashboardPage.js
│   └── ProfilePage.js
├── index.js
└── app.js
```

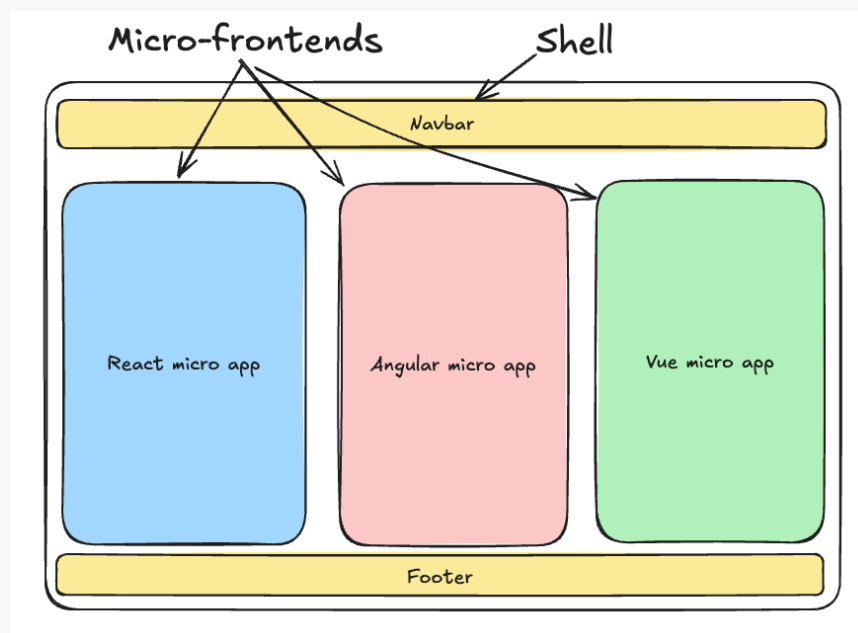
Spaghetti monolith 🍝

## ▼ Micro Frontends (MFE)

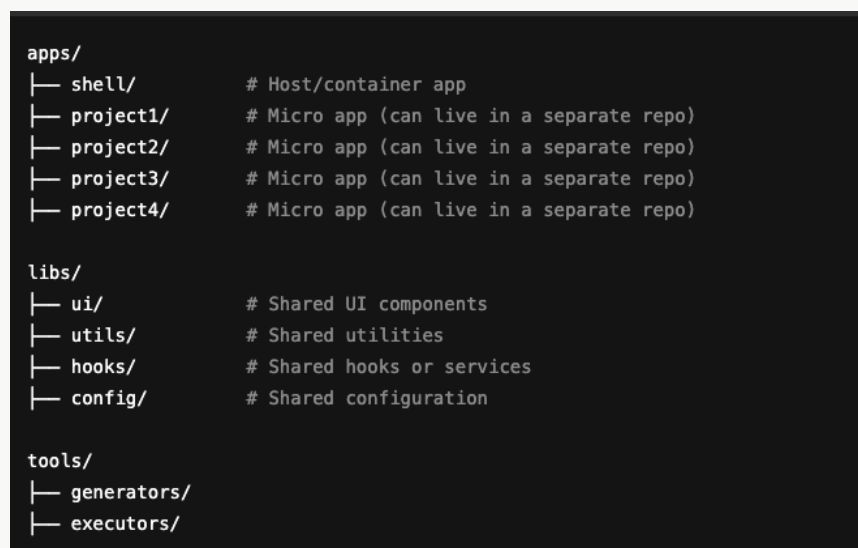




**Micro-frontend architecture** - bu Frontend loyihasini bir nechta qismlarga bo'lib, har birini alohida boshqarish imkonini beruvchi arxitektura. Ko'pincha katta loyihalarni qismlarga bo'lib, alohida jamoalarga topshirishda ishlatiladi. **Framework** tanlamaydi. Ko'p hollarda har bir loyiha alohida **Repository**'da bo'ladi. Ammo, bitta **Repository**'da ham ishlatsa bo'ladi.



Micro frontend architecture example



Micro frontend in Monorepo example

## Qanday turlari bor?

### ▼ **Iframe**

**Host** loyiha micro-loyihalarni Iframe orqali chaqiradi.

Kod misoli:

```
<iframe
  src="https://vector-academy.uz"
  width="100%"
  height="600"
  style="border: none"
></iframe>
```

### ▼ **Module-federation**

**Module federation** - **Webpack** tomonidan ishlab chiqilgan **module sharing** yechimi. Ko'pincha loyihalarni qismlarga bo'lib, alohida **build** qilib olib, **Runtime** paytida birgalikda ishlatiladi. **Host** va **Remote** tushunchalariga ega.

Loyihaga misol: <https://red-ocean-0fe4c4610.azurestaticapps.net/dashboard>

Source code: <https://www.npmjs.com/package/@angular-architects/module-federation-tools?activeTab=readme>

Native federation: <https://www.npmjs.com/package/@softarc/native-federation>

**Task:** *Native-federation* dan foydalanib kamida 2 ta freymvorkni bir joyda ishlating.

## ▼ Real-world examples

1. **Modular monolith.** Bitta loyiha.

```
src/  
├─ app/  
│   ├─ features/  
│   │   ├─ dashboard/  
│   │   └─ users/  
│   ├─ settings/  
│   ├─ notifications/  
│   └─ reports/  
│   └─ shared/  
│       ├─ components/  
│       ├─ services/  
│       └─ utils/  
│       └─ models/  
│   └─ core/  
│       ├─ guards/  
│       ├─ interceptors/  
│       └─ layout/  
│   └─ routes/  
├─ assets/  
└─ environments/
```

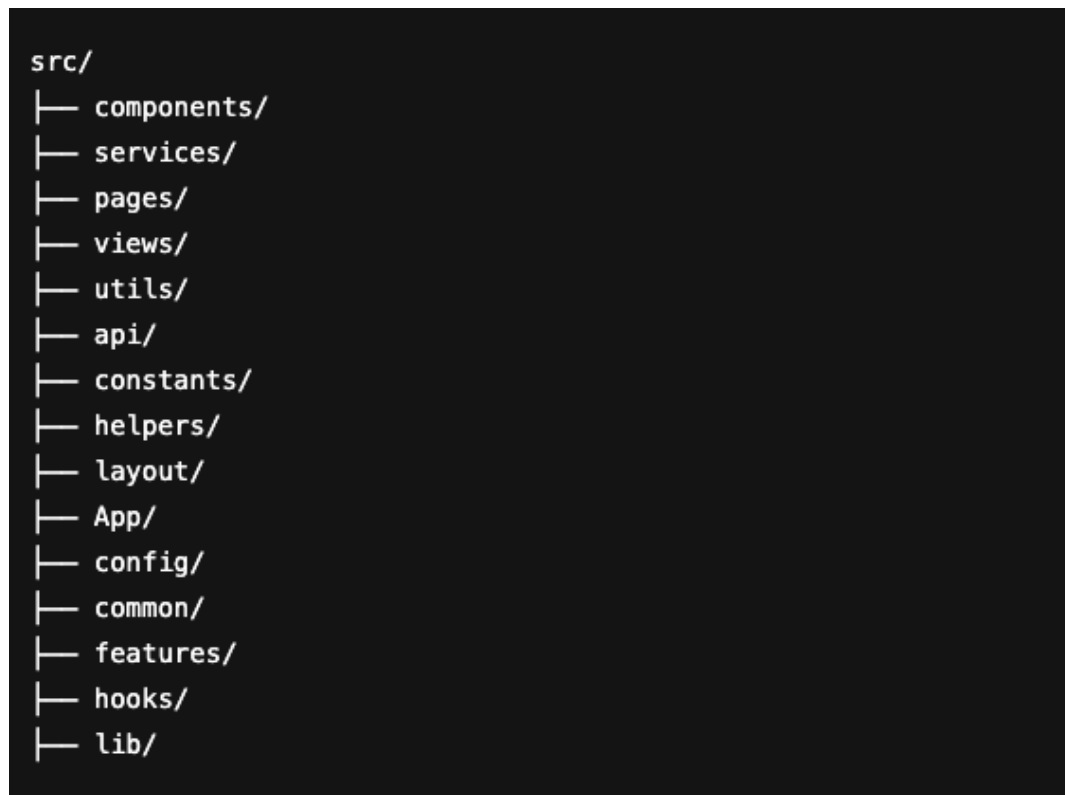
Modular monolith. Single project.

2. **Modular monolith.** Bir nechta loyihalarga ajratilgan. Ammo, barchasi bitta **Shell application** ichida chaqiriladi va birga **build/deploy** bo'ladi.

```
apps/  
├─ host-app/  
  
libs/  
├─ modules/  
│   ├─ project1/  
│   ├─ project2/  
│   ├─ project3/  
│   ├─ project4/  
│   └─ project5/  
├─ shared/  
│   ├─ ui/  
│   ├─ utils/  
│   └─ auth/
```

Modular monolith. Multiple projects

3. **Spaghetti monolith.**



Spaghetti style

#### 4. Micro-frontend.



Micro-frontend

### ▼ Web-components

**Web-component** - bu component qurishda **browser-native** usul. Xuddi **div**, **h1** teglar kabi Element. **Custom elements API** orqali, **Shadow DOM**’ga o’ralib quriladi.

Ko’pincha freymvork-tanlamas component’lar yozishda ishlatiladi.

— **Custom elements** — o’z HTML tegingizni yasash imkonini beruvchi **Browser API**.

— **Shadow DOM** — **Custom element**lar ichida ochilishi mumkin bo’lgan mustaqil va **private DOM tree**. Element stillarini **Encapsulate** qilishda foydalaniladi. Tashqaridan ham stil kirmaydi, ichkaridan ham.

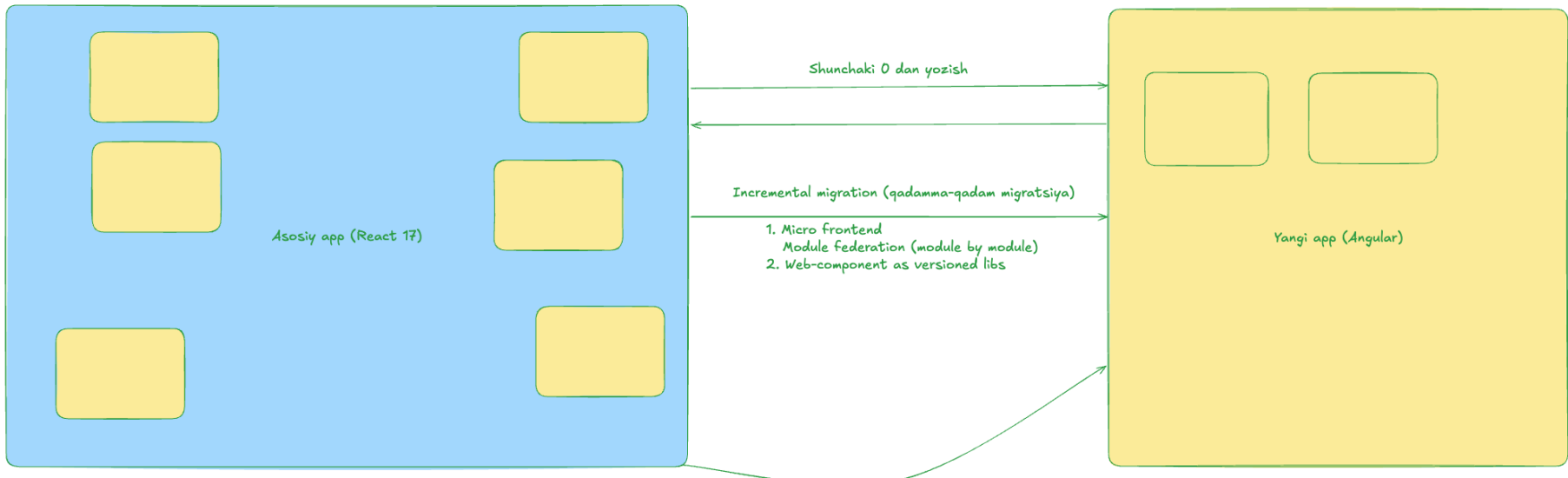
**Kutubxonalar:**

React - react-to-webcomponent, Lit, Stencil, vite-plugin-web-component

Angular - Angular elements, Lit, Stencil

Vue - @vue/web-component-wrapper, defineCustomElement API, Lit, Stencil

▼ Migrations



▼ Monorepo va Polyrepo

- 💡 **Monorepo** - bir nechta loyihalar bir **Repository**da saqlanishi (Ko'pincha **NX**, **Turborepo** kabi **tool**lardan foydalangan holda).
- Polyrepo** - har bir loyiha o'zining alohida **Repository**siga ega bo'lishi (**Micro-frontend** ishlatilgan hollarda ko'p uchraydi).

| # | Aspect              | Monorepo                                                                     | Polyrepo                                                                 |
|---|---------------------|------------------------------------------------------------------------------|--------------------------------------------------------------------------|
| 1 | Description         | Single repository that contains multiple projects/modules                    | Each project/module lives in its own separate repository                 |
| 2 | Purpose             | Centralized codebase to improve collaboration and consistency                | Decoupled codebases for flexibility and team autonomy                    |
| 3 | Code Sharing        | Easy — shared libraries, utils, design systems in same repo                  | Hard — requires publishing packages or linking across repos              |
| 4 | CI/CD               | Unified pipeline, affected-based builds, optimized workflows (e.g., Nx)      | Individual pipelines per repo; more isolated but harder to coordinate    |
| 5 | DX (Dev Experience) | Consistent tooling, easier refactoring, strong IDE support                   | Lightweight per repo, but more manual syncing of tools and configs       |
| 6 | Use Cases           | Modular monoliths, cohesive microfrontend setups, shared component libraries | Independent microfrontends, open-source components, large isolated teams |

Monorepo vs Polyrepo

▼ High level patterns

- ▼ Folder structure & export patterns
  - ▼ FSD (Feature sliced design)



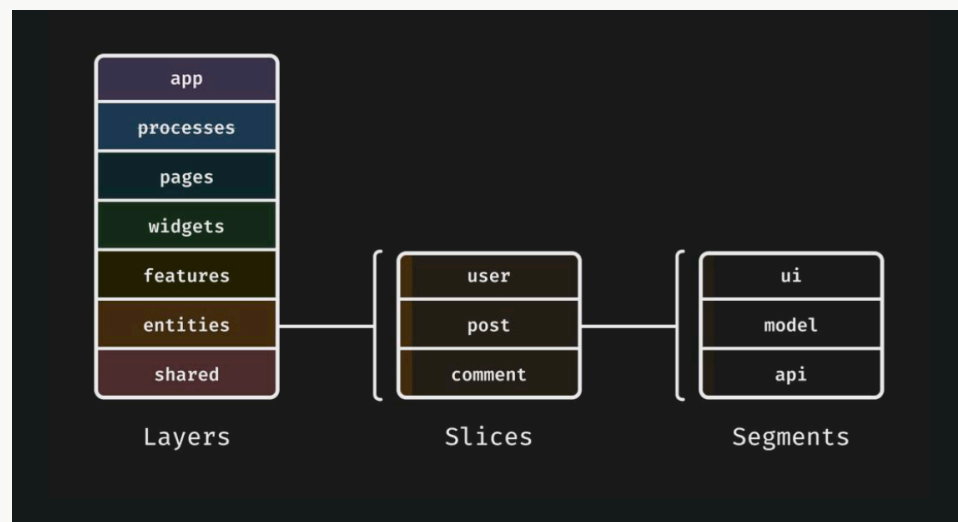
FSD (Feature sliced design) - bu Frontend architecture pattern hisoblanadi.

Maqsadi: Loyihadagi Folder structureni featurelarga bo'lib, ajratib chiqish.

O'rta-katta loyihalarda keng qo'llaniladi.

Core structure:

1. **Layers** - loyiha layerlarga bo'lingan bo'lishi kerak (*shared, core, features, pages* va hkz)
2. **Slices** - loyihani featurelarga bo'lib chiqish kerak ✂️.
3. **Segments** - har bir feature segmentlardan tashkil topishi (**UI, Data** va **Logic**).



FSD scheme

```
src/
├─ app/                                # App entrypoint: routes, layouts, injectables
│  └─ layout/
│     └─ main-layout/
├─ pages/                              # Routed screens (1:1 with routes)
│  └─ dashboard-page/
│  └─ exchange-page/
├─ widgets/                            # Reusable UI blocks (composed from features/entities)
│  └─ header/
│  └─ user-info-card/
├─ features/                           # Business use-cases (self-contained logic + UI)
│  └─ login-form/
│  └─ register-form/
│  └─ place-order/
├─ entities/                           # Domain logic (models, state, API) shared across features
│  └─ user/
│  └─ currency/
├─ shared/                             # Generic code (no business logic)
│  └─ components/                     # Dumb UI (Button, Input, Modal)
│  └─ utils/                          # formatters, validators, date utils
│  └─ http/                           # Http wrapper, interceptors, error handlers
│  └─ tokens/                         # DI tokens, feature configs
│  └─ types/                          # Global types/enums
└─ assets/                            # Static files (icons, i18n, logos, fonts)
```

## ▼ Barrel file pattern



**Barrel file pattern** - bu bir nechta **export**'larni bir joyga jamlab beruvchi fayl.

**Use case**'lar:

1. Biror feature o'zidan 2 ta modul **export** qilib, qolgan exportlarini yashirishi uchun juda qulay.

```
/src/app/features/user-profile
├─ user-profile.component.ts    ← exported
├─ user-profile.service.ts      ← exported
├─ user-profile.constants.ts    ← internal
├─ user-profile.mapper.ts       ← internal
├─ user-profile.utils.ts        ← internal
└─ index.ts                     ← barrel file
```

```
// ONLY exporting public API of the feature
export * from './user-profile.component';
export * from './user-profile.service';
```

2. **Shared** komponentlar guruhini jamlash uchun.

```
/src
├─ /shared
│   └─ /components
│       ├── /form
│       │   ├── Input.tsx
│       │   ├── Checkbox.tsx
│       │   └─ index.ts    ← form barrel
│       ├── /layout
│       │   ├── Header.tsx
│       │   ├── Footer.tsx
│       │   └─ index.ts    ← layout barrel
│       ├── Button.tsx
│       ├── Modal.tsx
│       └─ index.ts        ← components barrel
└─ index.ts                ← shared barrel
```

Ushbu ko'rinishdagi importlar shuning natijasi:

```
import { Button } from "@shared";
```

## ▼ UI Design patterns

### ▼ MVC

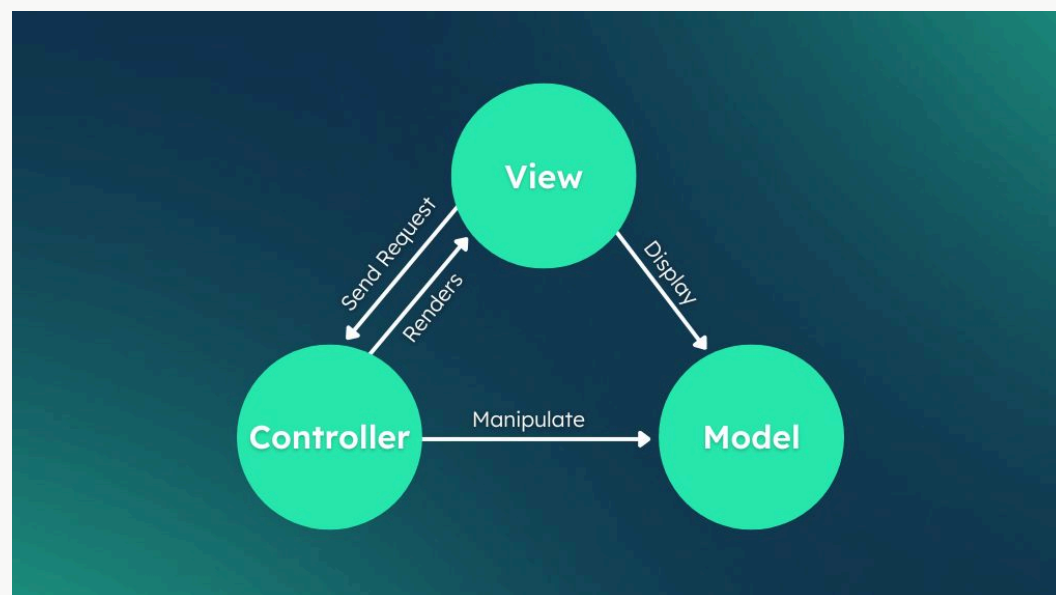




**MVC (Model - View - Controller)** - loyihani **UI (View)**, **Business logic(Model)** va **App logic(Controller)**ga bo'lib, ular orasida ma'lumot alishish uchun o'ylab chiqilgan **Architectural Design Pattern**.

Backend texnologiyalariga mosroq bo'lgani uchun, ko'pincha **PHP** (Laravel), **Java** (Spring MVC), **.NET** (ASP.NET MVC), **Ruby** (Ruby on Rails) kabi texnologiyalar foydalanadi. Hozirda Frontend freymvorklar **MVC**dan emas, o'zlariga mos bo'lgan **Pattern**lardan foydalanishadi (**Angular - MVVM**, **Vue - MVVM**, **React - View-State**).

1. **View** - Foydalanuvchiga ko'rinadigan qismi. Modeldan olingan ma'lumotlarni ko'rsatadi.
2. **Model** - Biznes logika saqlanuvchi joy. Masalan, ma'lumotlarni **Database**ga saqlash, olib berish yoki biznesga oid cheklovlarni tekshirish kabi vazifalarni o'z ichiga oladi.
3. **Controller** - Boshqaruvchi. **View**dan ma'lumot olib **Model**ga, **Model**dan o'zgarish olib **View**ga yuborib turuvchi qism. Dasturni **View** va **Model** bilan ulab, to'g'ri ishlashini nazorat qiladi.



MVC

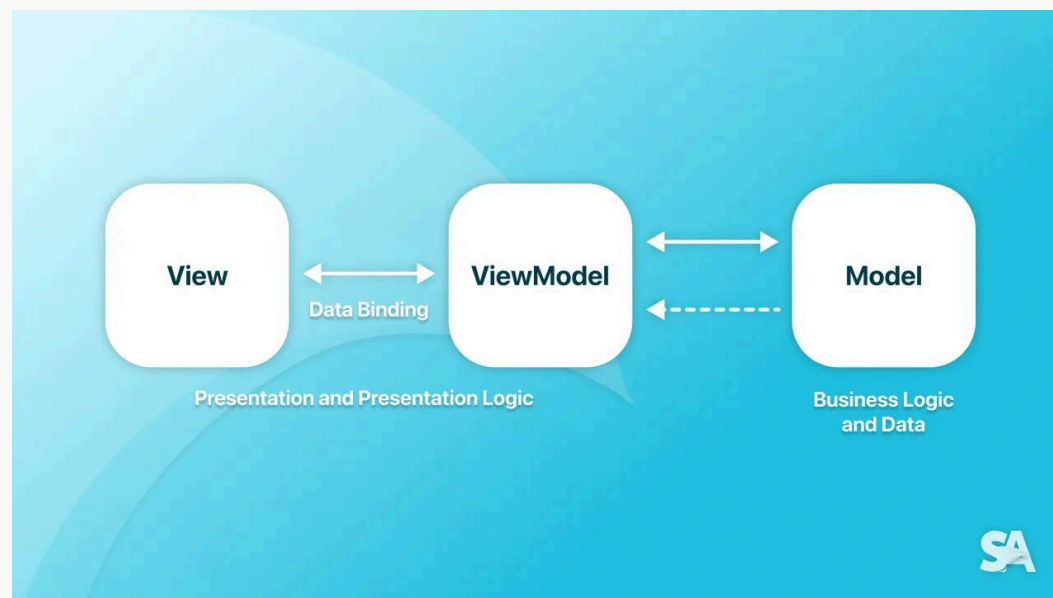
## ▼ MVVM



**MVVM** — bu loyihani **Business Logic (Model)**, **UI (View)** va **UI logic / State management (ViewModel)** qismlarga ajratishga mo'ljallangan **Architectural Design Pattern** bo'lib, **Frontend frameworklar (Angular, Vue, Solid)** uchun juda mos keladi.

**MVVM**da **View** bevosita **Model** bilan emas, balki **ViewModel** orqali bog'lanadi.

1. **Model** - bu asosiy biznes logika va ma'lumotlar ulashuvchi qism. **API** bilan ishlaydi.
2. **View** - bu foydalanuvchi ko'ruvchi qism. **ViewModel** bilan "binding"lar orqali gaplashadi. **Model**dan ajralgan.
3. **ViewModel** - **UI** uchun kerak bo'lgan barcha **Statelarni Model** bilan ishlagan holda o'zida saqlaydi. **Model** va **View** o'rtasidagi ko'priq bo'ladi. Ma'lumotlarni alishib turadi.



MVVM

## ▼ Atomic design methodology





**Atomic design** - bu **Design system** qurishda **UI component**larni o'z vazifasiga ega 5 ta kichik qismlarga ajratishni targ'ib qiluvchi metodologiya.

- **Atoms** - eng kichik **UI element**lar. Masalan **input**, **button**, **icon**, **label**.

```
<App-input />
<App-button />
```

- **Molecules** - Biror maqsad yo'lda **Atom**larni jamlangani. Masalan **form**, **accordion**.

```
<form> ← Molecule
  <app-label for="inputText1" text="Text 1" /> ← Atom
  <app-input id="inputText1" value="Text" />

  <app-button>Submit</app-button>
</form>
```

- **Organisms** - **Molecules** va **Atoms**dan tashkil topgan **UI block**. Masalan, **LoginComponent**, **HeaderComponent**.

LoginComponent ↓

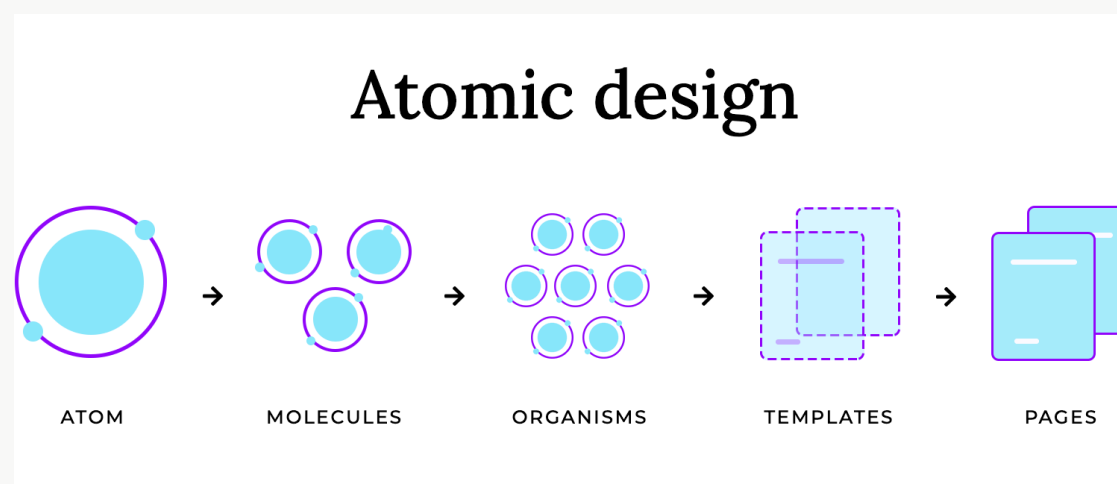
```
<form> ← Molecule
  <app-label for="inputText1" text="Text 1" /> ← Atom
  <app-input id="inputText1" value="Text" />

  <app-button>Submit</app-button> ← Atom
</form>

<hr /> ← Atom

<div> ← Molecule
  <span> Login with Google </span>
  ....etc
</div>
```

- **Templates** - Sahifani strukturasi. **Dumb component** desa bo'ladi. **Data**ni **Page**dan oladi.
- **Pages** - Asosiy sahifa. Route ulanadigan qism. **State**ni boshqaradigan va pastga uzatadigan qism.



## ▼ Container-Presentational Pattern



UI'ni **ikki turdagi komponentlarga** ajratish orqali **clean architecture** hosil qilish uchun ishlatiladigan pattern:

- **Smart components (Container)** — business logic, state, API, routing
- **Dumb components (Presentational)** — only inputs/outputs, pure UI, reusable

**Masalan:**

**LoginPage** → **LoginFormComponent**

**LoginPage** — Ma'lumotlarni tepadan LoginFormComponent'ga beradi va Forma submit bo'lganda tayyor natijani oladi va API'ga yo'naltiradi.

**LoginFormComponent** - tepadan ma'lumot olib, tepaga ma'lumot chiqaradi.

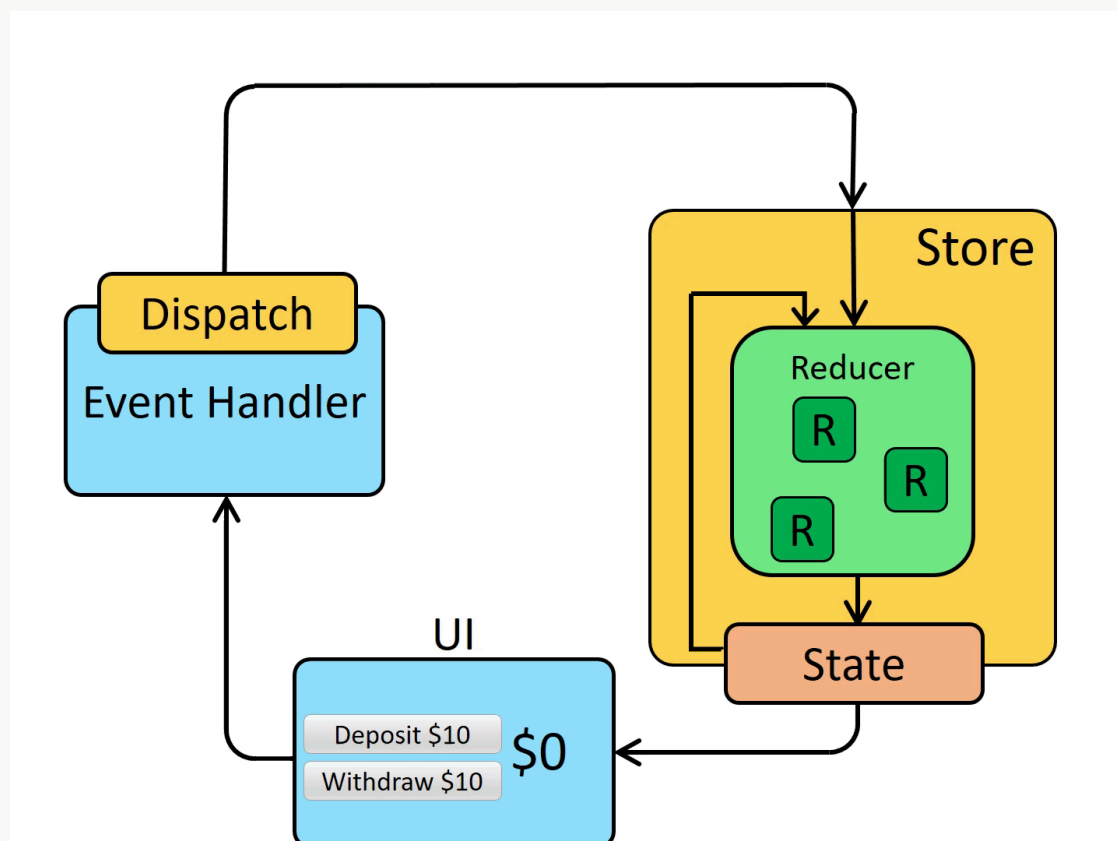
## ▼ State management patterns

### ▼ Redux pattern



**Redux pattern** - Loyihada state'larni boshqarishda **Unidirectional data flow** va **Flux pattern** asosida ishlab chiqilgan, "central store" tushunchasiga asoslangan **State management pattern**.

**Tools:** Redux, NgRx, Vuex

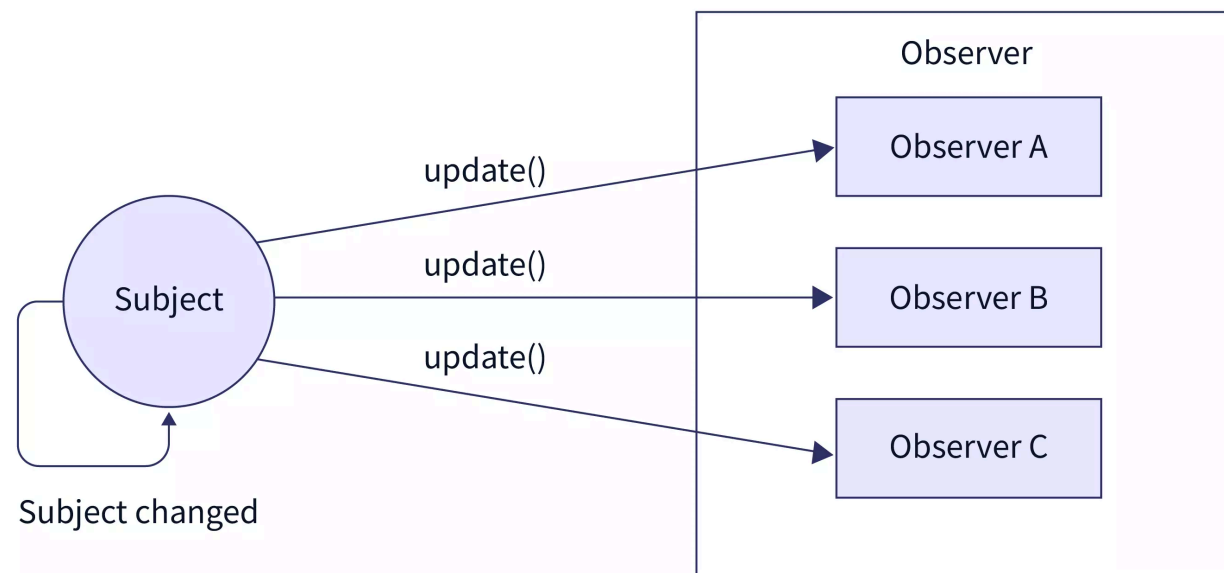


### ▼ Observer Pattern



**Observer Pattern** - qiymat o'zgarsa, uni eshituvchilarni(subscribers) avtomatik yangilash uchun ishlatiladigan **Pattern**. Ko'pincha **bidirectional** bo'ladi.

**Tools:** RxJS, Signals, MobX, Vue 2 reactivity



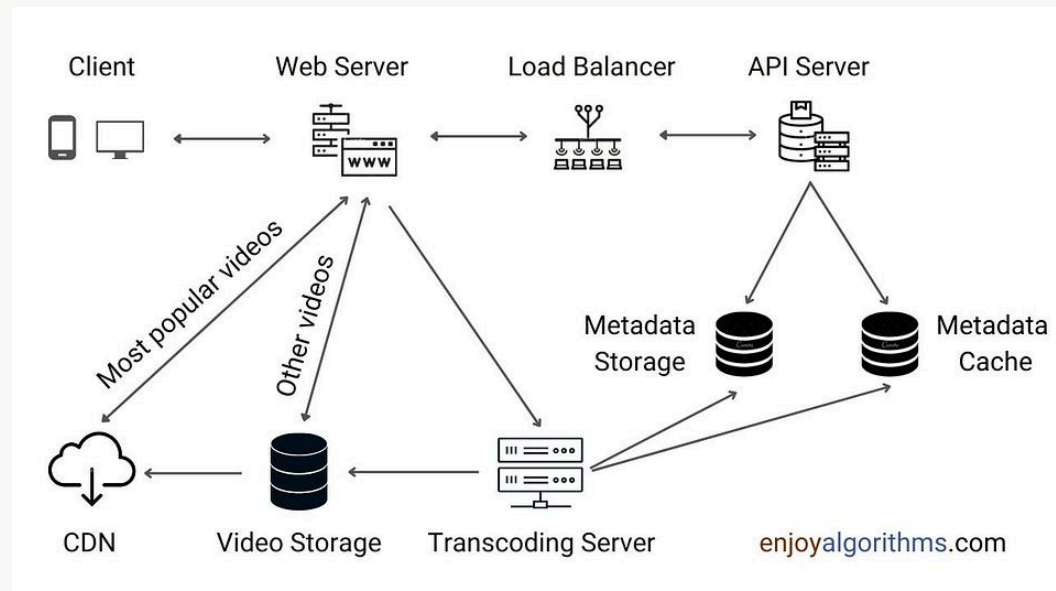
SCALER  
Topics

## ▼ System design



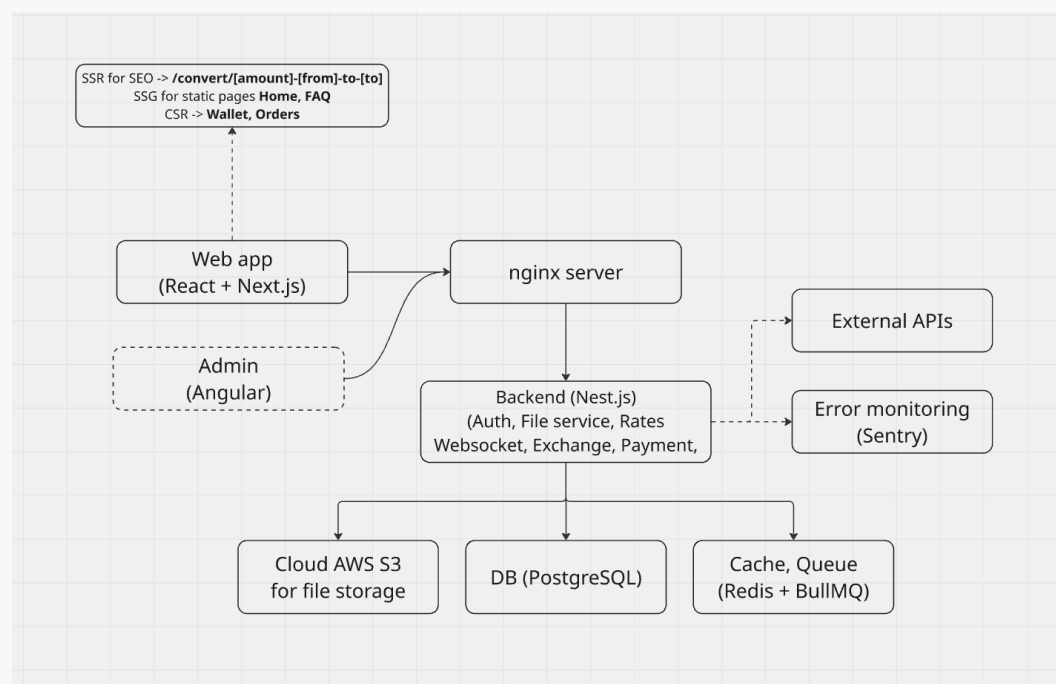
**System design** - butun boshli loyihani ishlashini rejalashtirish degani. **Frontend**, **Backend**, **DB** va ular atrofidagi barcha logikalar bir-biri bilan qanday ishlashi va optimizatsiya qilish kabi masalalarni hal qilinadi.

Masalan, **Youtube**'ning **System design** ko'rinishi:



O'z loyihamizni **System design** qilishga harakat qilamiz.

**Dollar** va **so'm** almashtirish sayti:



**Resource to learn:** [https://www.youtube.com/watch?v=F2FmTdLtb\\_4](https://www.youtube.com/watch?v=F2FmTdLtb_4)

## ▼ Frontend system design

**Frontend system design** tuzishda 3 xil talablarni inobatga oling: Loyihani **Functional** va **Technical** talablari.

**Functional**'ga misol – Notifications, Messaging, Like and Comment, Subscribe, Upload profile image.

**Technical**'ga misol – SEO friendly, Fast initial loads, A/B testing support, Scalable code, Multiple Frontends, Accessibility, Real-time communication.

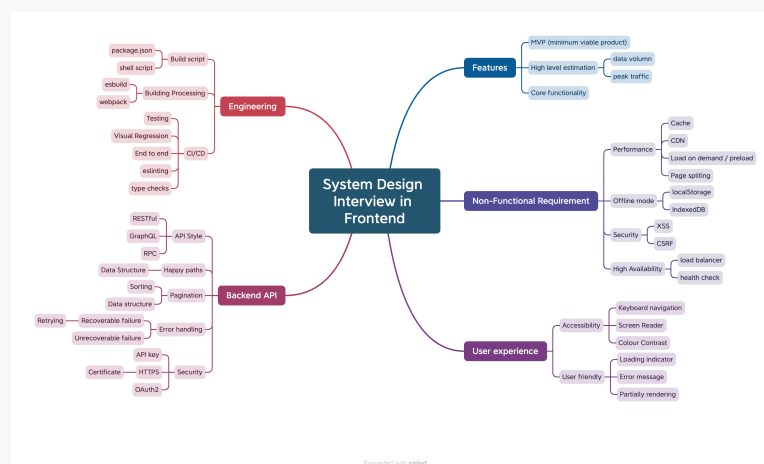
### Frontend system design topics:

1. Architecture pattern (Monolith, Modular monolith, Microfrontend)
2. Component architecture (Smart/dumb components)
3. Rendering + Routing (CSR, SSR, SSG, ISR, SPA)
4. State management
5. API layer
6. Testing

7. Design system
8. Optimizations (Performance optimization, initial load optimization, build-time optimization, Performance Metrics...)
9. Security (Authentication, XSS, CSRF, JWT...)
10. Tooling (Monorepo tools, Linters, Formatters)
11. CI/CD (Build, Deploy, Rollback, Environments)
12. Folder structure
13. DX
14. Documentation (Docs, Storybooks...)
15. i18n
16. Cross Platform UI (Responsiveness, Mobile-first UI..., PWA)
17. Release strategy

### Resources:

1. [https://www.youtube.com/playlist?list=PLv8UD0INnLNKqg5iXdtIVOhEaegts9F\\_Z](https://www.youtube.com/playlist?list=PLv8UD0INnLNKqg5iXdtIVOhEaegts9F_Z)



1. <https://www.youtube.com/@d.zhiganov/playlists>

### Intervyuda javob berish uchun ketma-ketlik:

1. Requirement (Functional/Technical requirements)  
How deep?
2. High-level design
3. Feature design
4. Component design
5. Communications

## ▼ Git

### ▼ Branching strategies

**Branching strategies** — loyihadagi **branch**'larni qanday tartibda boshqarish usullari. Ko'pincha **Git flow**'dan foydalaniladi.

| Strategy         | Main Branches                                                                                                      | Key Concepts                                                                              | Pros                                        | Cons                                      | Best For                                                    |
|------------------|--------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|---------------------------------------------|-------------------------------------------|-------------------------------------------------------------|
| Git Flow         | <code>main</code> , <code>develop</code> , <code>feature/*</code> , <code>release/*</code> , <code>hotfix/*</code> | Structured release cycle with clear branch purposes                                       | Good for planned releases, clear roles      | Heavy and complex for fast-moving teams   | Projects with long-term planning and strict release process |
| GitHub Flow      | <code>main</code> , feature branches                                                                               | Everything merged into <code>main</code> via PRs; deploy from <code>main</code>           | Simple, continuous delivery, good for CI/CD | No support for pre-release testing flows  | Startups, CI/CD pipelines, rapid delivery                   |
| GitLab Flow      | <code>main</code> , env branches (e.g. <code>pre</code> , <code>prod</code> ), feature branches                    | Combines GitHub Flow + env-based deployment                                               | Flexible, handles staging/production        | Can be confusing without discipline       | Teams deploying to multiple environments                    |
| Trunk-Based      | <code>main</code> (aka trunk)                                                                                      | Short-lived branches; direct commits or PRs into <code>main</code> ; deploy often         | Simple, fast iterations, aligns with DevOps | Needs strong CI/CD; risky without testing | High-performing teams, modern DevOps setup                  |
| One-Flow         | <code>main</code> , temporary <code>release/*</code>                                                               | A simplified Git Flow without <code>develop</code> ; uses temporary release branches only | Clean, less branching overhead              | Requires discipline in tagging/releases   | Teams wanting structure without Git Flow complexity         |
| Release Train    | <code>main</code> , <code>release/*</code> , <code>hotfix/*</code>                                                 | Time-based releases; features merged by schedule, not readiness                           | Predictable delivery, scalable              | Features might wait for next train        | Enterprises needing regular, predictable release cycles     |
| Forking Workflow | Forked repos, feature branches                                                                                     | Each developer forks and PRs to original repo                                             | Great for open-source, contributor control  | Overhead of forks, slower feedback        | Open-source projects, external contributions                |

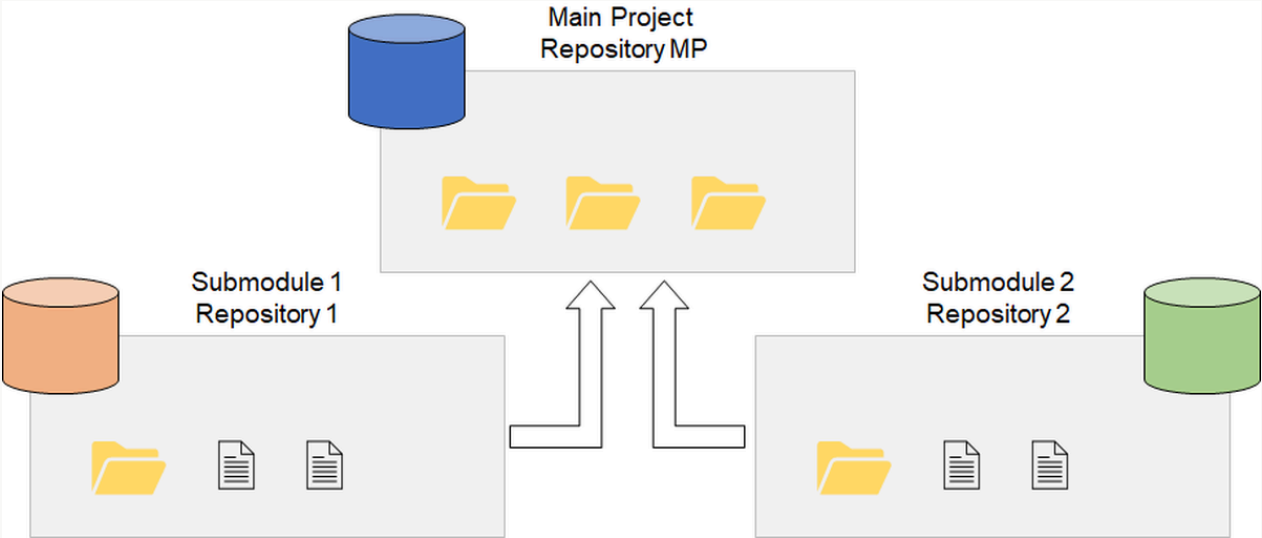
### ▼ Submodules



**Git submodules** - Bir git repo ichida boshqa git repo'larni ulab ishlatish. Ulangan repo'lar **submodule** deyiladi.

#### Submodule qo'shish

`git submodule add https://github.com/some/repo.git path/to/submodule`



### ▼ Design system



**Design system** — **UI components, colors, spacing, fonts, theming** — hammasi yagona tizimda bo'lsa, shu tizimni **Design system** deymiz.

#### 3 ta asosiy qismi:

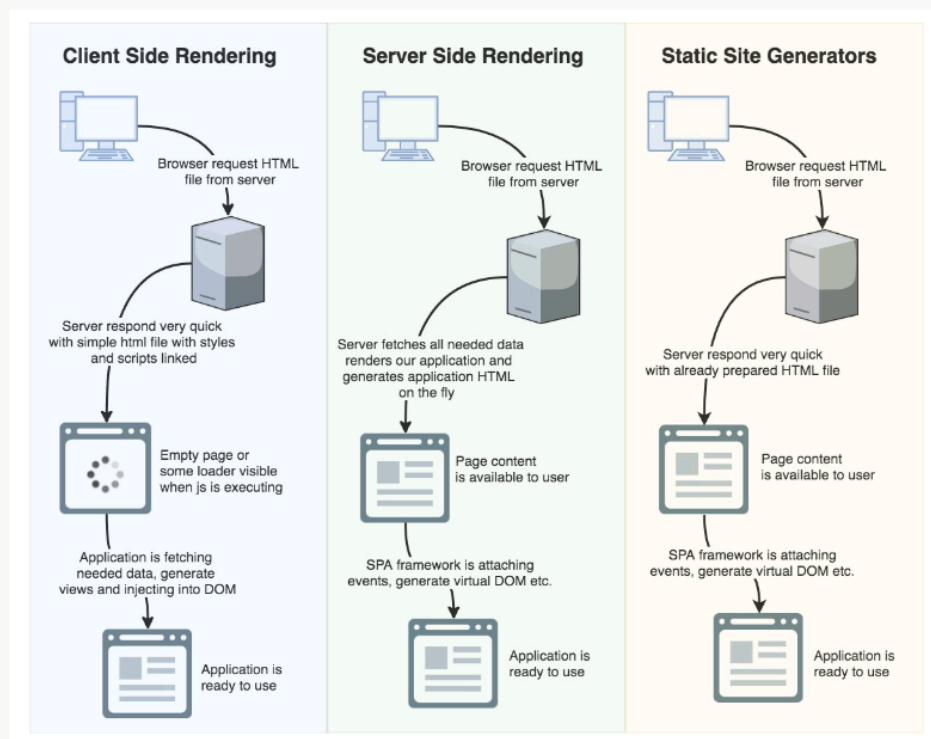
1. **Design tokens** — colors, typography, animations, spacing, icons (`$color-primary: #123456`; `$spacing-sm: 4px`)
2. **Components** — UI komponentlar
3. **Documentation** — dizayn sistemadan qanday foydalanish haqida dokumentatsiya.

### ▼ Rendering turlari





**Rendering** deb bizning **HTML**, **CSS** va **JS** kodlarimizni foydalanuvchi ko'ra oladigan holatga o'girishga aytiladi.



Rendering types

## ▼ Custom library



**Custom library** — loyihalar orasida qayta-qayta ishlatilayotgan kodlarni bitta kutubxona ichiga joylash usuli. 2 ta turli bo'ladi. **Public** va **private**.

[[Live coding time]]

## ▼ Versioning va Release strategiyalari



**Versioning strategy** — loyiha versiyalarini qanday tarzda olib yurishingiz. Masalan:

1. **Semantic versioning (SemVer)** — MAJOR.MINOR.PATCH (2.3.1). Kutubxonalar yoki Backend API servislar uchun juda zo'r yechim.

1.1. **Major** — Breaking changes, katta o'zgarishlar. Masalan, method nomlari o'zgarishi, API'lar o'zgarishi va hokazo.

1.2. **Minor** — Yangi **feature**'lar yoki loyihani buzmaydigan o'zgarishlar (**Backward compatible**). Masalan, yangi component qo'shish yoki funksiya qo'shish.

1.3. **Patch** — Kichik **bug fix**'lar, **refactoring** va shu kabi kichik o'zgarishlar. Masalan, uzun funksiyalarni qismlarga ajratish, CSS'da kichik to'g'irlashlar va hkz.

2. **Date-based versioning** — sana va vaqt bilan qilish. Tez o'zgarishlar bo'ladigan katta loyihala uchun yaxshi.

**Tools:** npm version, semantic-release, lerna, git tags

**Release strategy** — loyihani foydalanuvchiga yetkazish strategiyalari.

Mashxur uslublar:

1. **Big bang release** — hamma kodni bittada **ship** qilish 🚀

2. **Canary release** — Yangi **kod**larni kamroq odamga ko'rsatishdan boshlash.

5% → 10% → 20% → 50% → 100%

3. **A/B Testing Release** — Yangi versiya bilan eski versiyani ko'rsatishni foydalanuvchilarni teng yarmiga bo'lish: 50/50

## ▼ Testing turlari

### ▼ Manual testing



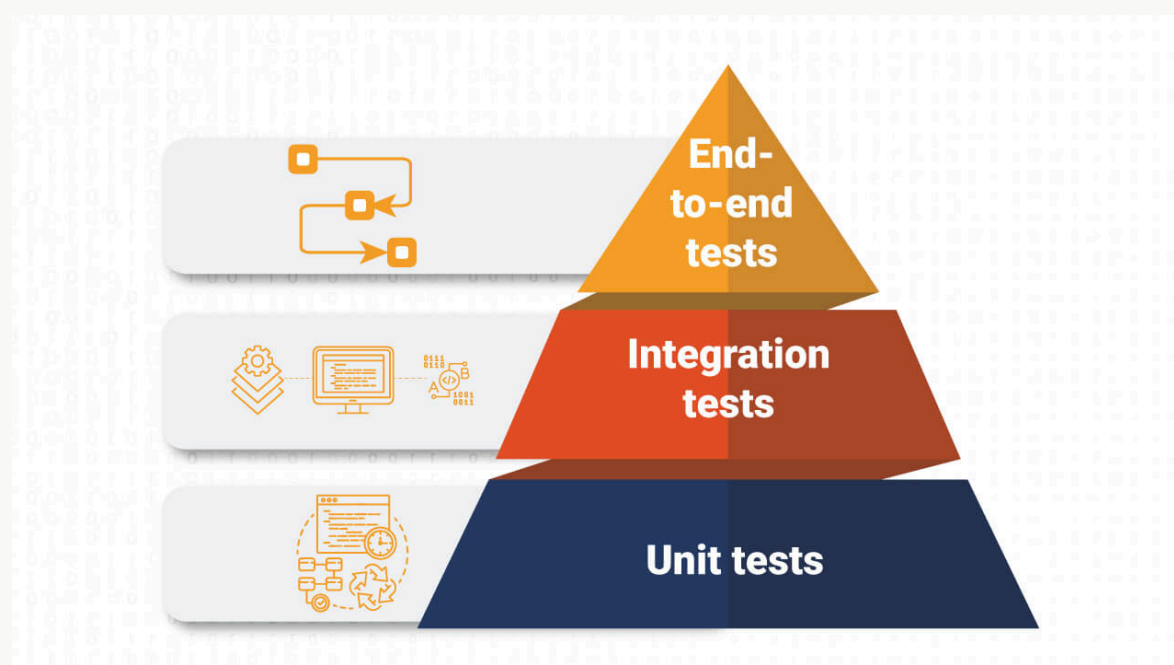
**Manual testing** — **tester**lar tomonidan qo'lda saytni test qilish.

### ▼ Automation testing



**Automation testing** — dastur kodini yoki ishlayotgan dastur o'zini avtomatik test qilish. 3 ta asosiy turi mavjud. **Unit**, **Integration** va **E2E**.

Bular **Testing pyramid** deyiladi.



### ▼ Unit tests





**Unit tests** — loyihadagi eng mayda, ammo eng muhim qism bo'lgan funksiyalar, **utility**'lar, servislar, **UI component**larni test qiladi.

Action → Result

App-button →

- should exist
- should output event when clicked
- should output event when double-clicked
- should disable button if disabled is true
- should show spinner when loading is true
- should hide spinner when loading is false
- should change the text when passed
- should set primary styles when variant=primary
- should set button type when type is passed (default button)

Masalan,

```
describe('Function or Component Name', () => {
  it('should do something', () => {
    // Arrange
    const input = ...

    // Act
    const result = someFunction(input)

    // Assert
    expect(result).toBe(expected)
  });
});
```

**Tools:** Jest, Mocha, Chai, Vitest, Jasmine, Karma...

## ▼ Integration tests



Loyihadagi birnechta qismlarning (**module**, **service**, **component**...) bir-biri bilan integratsiyasini test qiladi.

**Misollar:**

1. Form validation + Submit
2. Action + API call + UI update
2. Theme change + UI update
3. Action + Alert message

## ▼ E2E tests



**End to End testing** — foydalanuvchini simyulatsiya qiladigan testlar. Masalan,  
"Login qil → Profil dropdown'ni bos → Theme'ni o'zgartir → UI o'zgardimi?"

```
describe('Login Flow', () => {
  it('should login and redirect to dashboard', () => {
    cy.visit('http://localhost:4200/login'); // or localhost:3000 or your app port

    // fill out the login form
    cy.get('input[name="email"]').type('umar@example.com');
    cy.get('input[name="password"]').type('secret123');
    cy.get('button[type="submit"]').click();

    // after login
    cy.url().should('include', '/dashboard');
    cy.contains('Welcome, Umar').should('be.visible');
  });
});
```

**Tools:** Cypress, Playwright, Selenium

## ▼ Optimization turlari va usullar

### ▼ Initial load Optimization



**Initial load optimization** texnikalari:

1. **Bundle optimization** — loyiha kodlari jamlangan fayllarni hajmini qisqartirish.

**Usullar:**

- Kutubxonalarni tekshirish va qisqartirish (**source-map-explorer**, **webpack-bundle-analyzer**)
- Tree-shaking
- Minification

2. **Compression** (GZIP yoki Brotli)
3. **Lazy loading** va **Preloading strategy**
4. **Caching** (browser's cache, cache-control, CDN, PWA)
5. **Defer non-critical CSS/JS**
6. **Image optimization** (WebP, AVIF)

### ▼ Performance optimization



1. **Rendering & Reactivity optimization** (Framework specific)
2. **Layout Thrashing** ishlatmang. (loop ichida DOM interaction)  
**Solution:** requestAnimationFrame, batching
3. requestIdleCallback(cb(({ timeRemaining, didTimeout }) => {}), { timeout })
4. **Reflow** va **repaint**

## ▼ Web security



**XSS(Cross site scripting)** - Saytga begona bo'lgan script'larni kiritish orqali saytni buzishga urunish. Yechimlar: **DOMPurify**, **HTMLSanitizer** va hkz.

**CSS Injections** - Dasturga begona bo'lgan biror script'ni CSS orqali kiritish usuli. Ko'pincha **background-image: url("begonaUrl");** ni kiritish orqali bo'ladi. Hozirda yangi versiyali browser'lar allaqachon bu muommoni oldini olgan.

**SQL Injections**

**Iframes** - HTML'dagi **Iframe** tegini ishlatilgan holatda, saytimizga hujum qilish uchun yana bir imkon qoldirgan bo'lamiz. Bundan hacker'lar **clickjacking** hujumini qilishi ham mumkin.

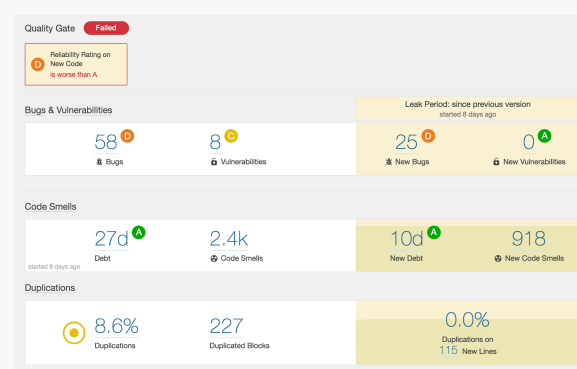
etc.

Maslahatlar

- Ishonchli **third-party** kutubxonalardan foydalaning.

- "Vulnerability"larni oldini oling. (**npm audit** va **Sonarqube** yoki shu kabi boshqa **Vulnerability scanners**'lardan foydalaning).

Example report:



Useful links

OWASP TOP 10: <https://owasp.org/www-project-top-ten/>

## ▼ Best practices, principles

### ▼ SOLID



### ▼ DRY



### ▼ KISS



### ▼ YAGNI



### ▼ SoC

